A INTERNSHIP REPORT ON

ARTIFICIAL INTELLIGENCE

Submitted in partial fulfillment of the requirements

for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

M HEMESH 22751A0584

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES, CHITTOOR-517127, A.P.

(Autonomous-NAAC Accredited)

(Approved by AICTE & Affiliated to JNTUA, Ananthapuramu)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(2025-26)

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES, CHITTOOR-517127, A.P.

(Autonomous – NAAC Accredited)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the internship report on **"ARTIFICIAL INTELLIGENCE"** is a genuine work of

M HEMESH 22751A0584

submitted to the department of Computer Science and Engineering, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, during the academic year 2025-26.

Signature of the Supervisor
Mr. P PRAVEEN, MCA., M. Tech.,
Assistant Professor,
Department of CSE,
Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Signature of the Head of Department
Dr.R KARUNIA KRISHNA PRIYA, M.Tech., Ph.D.,
Associate Professor & HOD,
Department of CSE,
Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Submitted for Semester End Examination held on ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INTERNSHIP CERTIFICATE

SkillDzire

AICTE

CERTIFICATE OF INTERNSHIP

This is to Certify that Mr./Ms

M Hemesh

Enrolled in the **Computer Science and Engineering - 22751A0584**

From College **Sreenivasa Institute of Technology and Management Studies**

of university **JNTUA, Anantapur**

has Successfully Completed short-term Internship programme titled

Artificial Intelligence

Under SkillDzire from **30-May-2025** to **28-June-2025** Organized By **SkillDzire**

Certificate ID: **SDST-25-24636**

Issued On: **28-June-2025**

Authorized Signature

ABSTRACT

Artificial Intelligence (AI) is a rapidly evolving field that focuses on developing systems capable of performing tasks that typically require human intelligence. This project explores the fundamental concepts, techniques, and real-world applications of AI, emphasizing how intelligent algorithms can be used to analyze data, recognize patterns, and make autonomous decisions.During this work, various AI approaches such as machine learning, deep learning, and natural language processing were studied and implemented to understand their practical use in solving real-life problems. The project involved working with datasets, applying preprocessing techniques, training models using Python libraries like NumPy, Pandas, Scikit-learn, and TensorFlow, and evaluating their performance through accuracy metrics.The experience provided valuable insights into how AI can optimize processes, enhance decision-making, and contribute to innovations across industries. Through this experience, I gained a strong understanding of how AI models can be trained to solve real-world problems efficiently and accurately. It also enhanced my ability to apply theoretical knowledge to practical challenges, improving my skills in coding, problem-solving, and analytical reasoning.Overall, the project provided valuable insights into the growing impact of Artificial Intelligence in diverse fields such as healthcare, business, and automation. It served as a foundation for future learning and development in building intelligent systems that contribute to technological advancement.

INDEX

CHAPTER NO.	TABLE OF CONTENTS	PAGE NO.
1	**INTRODUCTION**	
	1.1 INTRODUCTION OF THE COMPANY AND COURSE	01
	1.2 INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND KEY CONCEPTS	02-05
	1.3 OBJECTIVES OF ARTIFICIAL INTELLIGENCE PROGRAM	05-06
2	**PROJECT DESCRIPTION**	
	2.1 STEPS TO IMPLEMENT	07
	2.2 TASK 1	08-14
	2.3 TASK 2	15-20
	2.4 TASK 3	21-25
3	**SUMMARY AND CONCLUSION**	
	3.1 PROJECT SUMMARY	26
	3.2 CONCLUSION AND FRAMEWORK	27
	REFERENCES	28

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.2.1	Uploading chat file	14
2.2.2	Chat analyzer and emotion detector	14
2.3.1	Spam vs. ham email classifier	20
2.4.1	Image classification model	25

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION OF THE COMPANY AND COURSE

SkillDzire is a prominent real-time learning platform dedicated to providing advanced skill development programs aligned with current industry needs in emerging technologies. Founded in 2020 and based in Hyderabad, India, the company aims to bridge the gap between academic knowledge and practical industry skills for students and graduates. The Artificial Intelligence (AI) Internship offered by SkillDzire is a specialized, short-term program designed to provide students with a practical and comprehensive understanding of the AI domain. The program's objective is to transform foundational theoretical knowledge into implementable technical skills required for real-world AI applications.

COURSE DETAILS :

TITLE	**:** Artificial Intelligence
COMPANY	: SkillDzire
DURATION	**:** 4 Weeks
TOPICS COVERED	**:** Artificial Intelligence
COURSE START	**:** May 30th 2025
COURSE END	**:** June 28th 2025
INTERN ID	**:** SDST-25-24636

ABOUT THE COMPANY:

COMPANY NAME	**:** SkillDzire
WEBSITE	**:** https://www.skilldzire.com
INDUSTRY	**:** EdTech and Professional Training & Development Industries
ADDRESS	**:** 4th Floor, Bizness Square, 16, HITEC City, Hyderabad, India – 500081.

1.2 INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Definition: Artificial Intelligence refers to the theory and development of computer systems able to perform tasks normally requiring human intelligence. It enables machines to mimic cognitive functions, learn from data, and adapt to new inputs without explicit programming. It's spans various disciplines including machine learning, deep learning, and natural language processing. Its ultimate goal is to build intelligent agents that can perceive their environment and take actions that maximize their chance of successfully achieving their goals.

Role of a Artificial Intelligence Developer: AI Developers work with technologies such as Machine Learning (ML), Deep Learning, Natural Language Processing (NLP), and Computer Vision to build smart applications. They collect and preprocess data, train and evaluate models, and continuously improve system accuracy and performance.

KEY CONCEPTS USED FOR TASK 1: CHAT ANALYZER AND EMOTION DETECTOR

1. Natural Language Processing (NLP)

NLP is the core technology behind analyzing chat data. It involves techniques like tokenization, stop-word removal, and lemmatization to process and understand human language. Using NLP, the system can extract meaningful patterns, word frequencies, and linguistic features from conversations.

2. Sentiment and Emotion Analysis

This concept focuses on identifying the emotional tone and psychological state expressed in messages. Sentiment analysis classifies text as positive, negative, or neutral, while emotion detection goes deeper to identify emotions such as joy, anger, sadness, or fear. This helps interpret the underlying feelings behind chat conversations.

3. Machine Learning (ML) Models for Text Classification

Machine learning algorithms such as Naïve Bayes, SVM, and deep learning models like LSTM or BERT are used to train the system to automatically predict emotions and sentiments from chat data. These models learn from labeled datasets and continuously improve their accuracy in detecting emotional and behavioral trends.

Libraries/Tools Used:

- **Programming Language:** Python
- **Libraries for Analysis:** Pandas, NumPy, Matplotlib, Seaborn, WordCloud
- **NLP Libraries:** NLTK, SpaCy, VADER, TextBlob
- **Deep Learning Frameworks:** TensorFlow, PyTorch, Hugging Face Transformers
- **Visualization Tools:** Matplotlib, Plotly, Seaborn

KEY CONCEPTS USED FOR TASK 2 : SPAM EMAIL CLASSIFIER

1. Text Preprocessing

Text preprocessing is a crucial step that prepares raw email data for analysis. It involves cleaning the email text by removing punctuation, numbers, URLs, and special characters, as well as converting text to lowercase and eliminating stop words. This ensures that only meaningful words are used for training the spam detection model, improving accuracy and efficiency.

2. Machine Learning (ML) for Text Classification

Machine learning algorithms such as Naïve Bayes, Logistic Regression, or Support Vector Machines (SVM) are trained on labeled email datasets. These models learn to recognize patterns and features that differentiate spam from legitimate emails, allowing the system to automatically classify new emails with high accuracy.

3. Feature Extraction and Model Evaluation

Feature extraction methods like Bag of Words (BoW) and TF-IDF convert email text into numerical form for model training. The classifier's performance is then measured using evaluation metrics such as accuracy, precision, recall, and F1-score to ensure its reliability and effectiveness in detecting spam.

Libraries/Tools Used:

- **Programming:** Python
- **Libraries**: Scikit-learn, NLTK, SpaCy, TensorFlow, PyTorch
- **Development:** Jupyter Notebook, Google Colab
- **Deployment:** Flask/Django for web apps, Streamlit for demo
- **Dataset Sources:** Enron, Spam Assassin, Ling-Spam datasets

KEY CONCEPTS USED FOR TASK 3: DEVELOPMENT OF AN IMAGE CLASSIFICATION MODEL FOR A CUSTOM DATASET

1. Convolutional Neural Networks (CNN's)

CNNs are the core of modern image classification systems. They automatically extract spatial and visual features from images using convolution, pooling, and fully connected layers. CNNs can learn complex patterns such as edges, textures, and shapes, making them highly effective for accurately classifying images into predefined categories.

2. Dataset Preprocessing and Augmentation

Before training, images are preprocessed to ensure uniformity and improve model performance. This includes resizing, normalization, and augmentation techniques such as rotation, flipping, and zooming. Data augmentation increases the diversity of training images, helping the model generalize better and reduce overfitting.

3. Transfer Learning and Model Evaluation

Transfer learning uses pre-trained models like VGG16, ResNet, or MobileNet to improve accuracy and reduce training time, especially when the dataset is small. The trained model is evaluated using metrics such as accuracy, precision, recall, and confusion matrix to assess its performance and reliability in real-world applications.

Libraries/Tools Used:

- **Python :** Main programming language used for implementation.
- **Google Colab** : for development and experimentation
- **GitHub** version control and project management
- **Streamlit / Flask** for deploying the demo application**.**

1.3. OBJECTIVES OF ARTIFICIAL INTELLIGENCE

The main objective of Artificial Intelligence (AI) in SkillDzire is to empower machines to perform tasks intelligently and make decisions similar to humans. AI aims to automate repetitive and complex processes, reducing manual effort and saving time. It helps in analyzing large datasets efficiently to provide meaningful insights. AI also enhances accuracy and problem-solving capabilities in various applications. Overall, the goal is to improve efficiency, productivity, and learning outcomes for users and organizations.

Key Objectives

1. Intelligent Decision Making

AI enables machines to think, learn, and make decisions similar to humans. It allows systems to process information, recognize patterns, and choose the best course of action. By simulating human reasoning, AI can handle tasks that require judgment and adaptability. This helps organizations and individuals make smarter and faster decisions.

2. Task Automation

A major objective of AI is to automate repetitive and routine tasks. By doing so, it reduces manual effort, minimizes errors, and saves time for humans. Automation allows employees to focus on more creative, strategic, and high-level problem-solving activities. Over time, AI-driven automation improves overall workflow efficiency.

3. Problem Solving

AI is designed to analyze complex problems and provide solutions efficiently. It can process multiple variables, simulate scenarios, and predict outcomes with high accuracy. AI algorithms help in areas like logistics, healthcare, finance, and engineering to tackle challenges faster than traditional methods. This enhances decision-making under complex or uncertain conditions.

4. Data Analysis

AI can handle and analyze vast amounts of data quickly and accurately. It identifies trends, patterns, and correlations that might be impossible for humans to detect manually. This capability supports predictive analysis, forecasting, and informed decision-making. AI helps businesses and researchers turn raw data into actionable insights.

5. Real-world Applications

Artificial Intelligence is widely used in everyday life and various industries to make processes faster, smarter, and more efficient. It helps in healthcare for diagnosis and treatment, in education for personalized learning, in finance for fraud detection and trading, in transportation for traffic management and self-driving vehicles, and in customer service through chatbots and virtual assistants. AI also supports agriculture, cybersecurity, e-commerce, robotics, and entertainment, improving productivity, decision-making, and overall user experience.

CHAPTER 2

PROJECT DESCRIPTION

During the 4-week AI internship at SkillDzire, I worked on developing intelligent systems using artificial intelligence techniques. The project involved analyzing datasets, designing AI models, and implementing solutions to solve real-world problems. I learned to apply machine learning and deep learning algorithms for predictive analysis and decision-making. The internship helped me gain practical experience in data preprocessing, model training, and evaluation. Overall, it enhanced my understanding of AI applications and improved my technical and problem-solving skills.

2.1 STEPS TO IMPLEMENT

1. **Problem Definition**

   Understand the problem you want to solve with AI.

   Define clear objectives and expected outcomes.

2. **Data Collection & Preparation**

   Gather relevant datasets from sources like CSV files, APIs, or online repositories.

   Clean, preprocess, and structure the data for analysis.

3. **Model Selection & Training**

   Choose suitable AI/ML algorithms (e.g., Random Forest, LSTM, CNN).

   Split the dataset into training and testing sets.

   Train the model using the training data.

4. **Model Evaluation & Optimization**

   Test the model on unseen data.

   Evaluate performance using metrics like accuracy, RMSE, or F1-score.

   Optimize the model by tuning parameters or improving features.

5. **Deployment & Prediction**

   Use the trained model to make predictions on new data.

   Integrate the AI solution into an application or workflow.

   Visualize results and interpret outputs for decision-making.

2.2 TASK 1: CHAT ANALYZER AND EMOTION DETECTOR

Purpose of the task

The purpose of the Chat Analyzer and Emotion Detector is to extract meaningful insights from chat conversations and understand the emotional state of users. With the exponential growth of digital communication platforms like WhatsApp, Messenger, and Telegram, analyzing chat data helps in identifying communication patterns, behavioral trends, and sentiment orientation. This system provides valuable applications in personal reflection, business analytics, social behavior research, and mental health monitoring. By detecting emotions such as joy, sadness, anger, fear, and surprise, it enables users and organizations to gain deeper insights into interpersonal interactions and emotional wellbeing.

Development approach

The purpose of the Chat Analyzer and Emotion Detector is to extract meaningful insights from chat conversations and understand the emotional state of users. With the exponential growth of digital communication platforms like WhatsApp, Messenger, and Telegram, analyzing chat data helps in identifying communication patterns, behavioral trends, and sentiment orientation. This system provides valuable applications in personal reflection, business analytics, social behavior research, and mental health monitoring. By detecting emotions such as joy, sadness, anger, fear, and surprise, it enables users and organizations to gain deeper insights into interpersonal interactions and emotional wellbeing.

Concepts reinforced

The project reinforces several important concepts in Artificial Intelligence and Machine Learning. It strengthens understanding of Natural Language Processing (NLP) for processing and analyzing human language, sentiment analysis for classifying text as positive, negative, or neutral, and emotion detection for identifying complex emotional states from conversations. It also deepens knowledge of machine learning and deep learning techniques used for text classification, along with data preprocessing and feature engineering methods that transform raw chat data into meaningful numerical representations.

Challenges faced

During the development of the Chat Analyzer and Emotion Detector, several challenges were encountered. Handling unstructured chat text proved difficult due to slang, abbreviations, emojis, and inconsistent grammar, making preprocessing complex. Accurately detecting emotions was challenging because a single message could express multiple or subtle emotions, leading to ambiguity in classification. Data imbalance also posed a problem, as some emotions appeared less frequently in the dataset, requiring techniques like oversampling or weighted loss to improve model performance. Selecting the appropriate machine learning or deep learning model and tuning hyperparameters for optimal accuracy was time-consuming. Additionally, training deep learning models on large datasets required significant computational resources, including GPU support, which sometimes limited experimentation and training speed.

Skills gained

Working on the Chat Analyzer and Emotion Detector project helped develop several important skills. It enhanced practical knowledge of Python programming and handling textual datasets. The project provided hands-on experience with machine learning and deep learning techniques for natural language processing, sentiment analysis, and emotion detection. Skills in data preprocessing, feature extraction, and model evaluation were strengthened, along with the ability to visualize insights using graphs, charts, and word clouds. Additionally, the project improved problem-solving abilities, logical thinking, and understanding of AI applications in analyzing human communication and emotions.

Learning outcome

By completing this project, the learner gains the ability to develop AI-based systems capable of analyzing human communication and detecting emotions from text. The project enhances problem-solving skills, logical thinking, and understanding of real-world AI applications. It also builds confidence in working with NLP techniques, text classification models, and data visualization, providing a strong foundation for advanced research or practical applications in AI, psychology, business analytics, and social sciences.

SOURCE CODE:

1. import os

```
from flask import Flask, request, render_template
import textwrap
from IPython.display import display, Markdown

import google.generativeai as genai
# Initialize Flask app
app = Flask(__name__)

# Configure GenerativeAI API key
genai.configure(api_key="AIzaSyA-tAJWZDUcDpMEo8IfT3wEI9D39KMKVV8")
# Function to convert text to Markdown
def to_markdown(text):
    text = text.replace('•', '  *')
    return Markdown(textwrap.indent(text, '> ', predicate=lambda _: True))

# Function to generate summary and emotions for the chat text
def generate_summary_and_emotions(chat_text):
    model = genai.GenerativeModel('gemini-pro')
    rply = model.generate_content("Summarize and give the emotions for the given WhatsApp
chat"+chat_text)
    to_markdown(rply.text)
    return rply.text
# Route for home page with file upload form
@app.route('/', methods=['GET', 'POST'])
def home():
if request.method == 'POST':
# Check if a file was uploaded  if
'Chat_file' not in request.files:
return render_template('error.html', message='No file part')
```

```
file = request.files['Chat_file']
# Check if the file is empty
if file.filename == '':
return render_template('error.html', message='No selected file') #
Check if the file is of allowed type
if file and file.filename.endswith('.txt'): #
Read the file content
chat_text = file.read().decode('utf-8')
# Generate summary and emotions
summary_and_emotions = generate_summary_and_emotions(chat_text) #
Render the result template with summary and emotions
return render_template('result.html', summary_and_emotions=summary_and_emotions) else:
return render_template('error.html', message='Invalid file type. Please upload a text file') return
render_template('index.html')
if __name___== '__main__
    app.run(debug=True)
```

2.

```
import pathlib
import textwrap
import google.generativeai as genai
# Used to securely store your API key
from IPython.display import display
def to_markdown(text):
  text = text.replace('•', '  *')
  return Markdown(textwrap.indent(text, '> ', predicate=lambda _: True))
genai.configure(api_key="AIzaSyBiJP5jccCgmtz3FaQz9UAOKo470f9EuGE")
model = genai.GenerativeModel('gemini-pro')
  with open('chat.txt', 'r', encoding='utf-8') as file:
```

```
chat_text = file.read()
rply = model.generate_content("Summarize and give the emotions for the given whatsapp
chat"+chat_text)
to_markdown(rply.text)
print(rply.text)
```

3.
```
import os
from flask import Flask, request, render_template
import textwrap
from IPython.display import display, Markdown
import google.generativeai as genai
# Initialize Flask app
app = Flask(__name__)
# Configure GenerativeAI API key
genai.configure(api_key="AIzaSyBiJP5jccCgmtz3FaQz9UAOKo470f9EuGE")
# Function to convert text to Markdown
def to_markdown(text):
    text = text.replace('•', '  *')
    return Markdown(textwrap.indent(text, '> ', predicate=lambda _: True))

# Function to generate summary and emotions for the chat text
def generate_summary_and_emotions(chat_text):
    model = genai.GenerativeModel('gemini-pro')
    rply = model.generate_content("Summarize and give the emotions for the given WhatsApp
chat"+chat_text)
    to_markdown(rply.text)
return rply.text

# Route for home page with file upload form
@app.route('/', methods=['GET', 'POST'])
def home():
```

```
    if request.method == 'POST':
      # Check if a file was uploaded
      if 'Chat_file' not in request.files:
          return render_template('error.html', message='No file part')
      file = request.files['Chat_file']
    # Check if the file is empty if
    file.filename == '':
    return render_template('error.html', message='No selected file')


# Check if the file is of allowed

      if file and file.filename.endswith('.txt'):
          # Read the file content
          chat_text = file.read().decode('utf-8')
       # Generate summary and emotions
          summary_and_emotions = generate_summary_and_emotions(chat_text)

       # Render the result template with summary and emotions
          return render_template('result.html', summary_and_emotions=summary_and_emotions)


      else:
          return render_template('error.html', message='Invalid file type. Please upload a text file')
   return render_template('index.html')


if __name___== '__main__':
   app.run(debug=True)
```

.

OUTPUT:

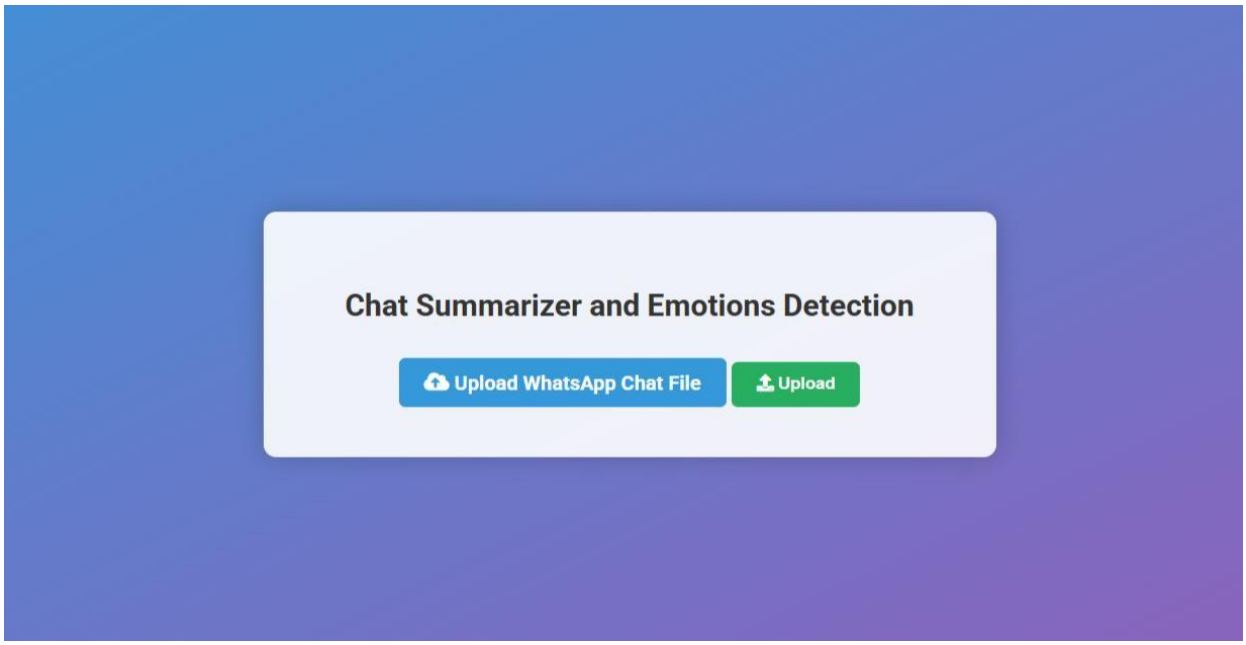

Fig 2.2.1: Uploading chat file

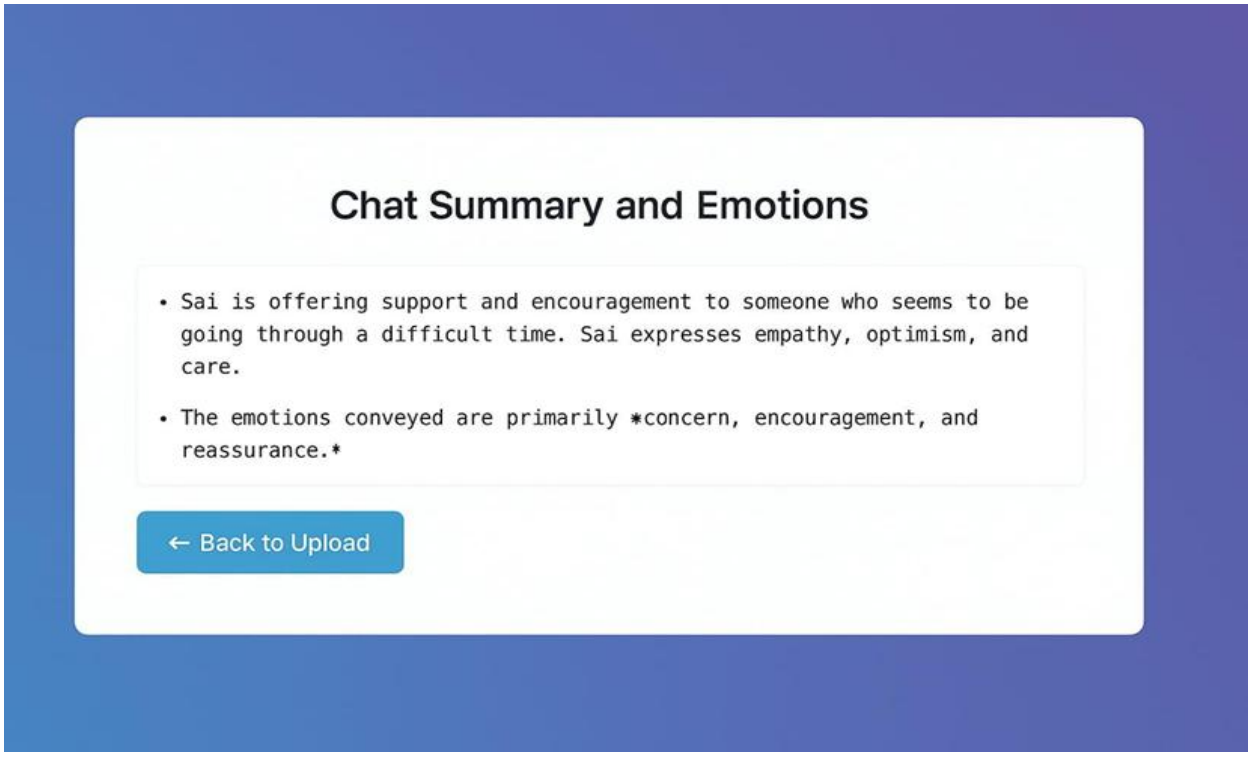

Fig 2.2.2: Chat analyzer and emotion detector

2.3 TASK 2: SPAM EMAIL CLASSIFIER

Purpose of the Task

The purpose of the Spam Email Classifier project is to automatically identify and filter out unwanted or malicious emails, improving user productivity and security. Spam emails are a common cybersecurity threat, leading to data theft, phishing attacks, and system vulnerabilities. This system leverages Artificial Intelligence and Machine Learning to detect spam by analyzing textual and semantic patterns, enabling users and organizations to manage their inboxes more efficiently and prevent potential security breaches.

Development Approach

The development of the Spam Email Classifier involves several structured steps. First, a dataset of emails labeled as spam or ham (legitimate) is collected from publicly available sources or email systems. The raw email text is preprocessed by removing punctuation, numbers, URLs, special characters, and stop words, and converting all text to lowercase. Next, features are extracted using methods like Bag of Words, TF-IDF, or word embeddings to numerically represent the textual content.

Concepts Reinforced

This project reinforces key concepts in Artificial Intelligence and Machine Learning. It strengthens understanding of text preprocessing techniques and feature extraction methods for converting raw text into numerical data suitable for machine learning. The project also deepens knowledge of classification algorithms such as Naïve Bayes, SVM, and Logistic Regression, as well as model evaluation using metrics like accuracy, precision, recall, and F1-score.

Challenges Faced

Several challenges were encountered during the development of the Spam Email Classifier. Handling unstructured email text with inconsistent formatting, special characters, and HTML tags required extensive preprocessing. Imbalanced datasets, where spam or ham emails were less frequent, affected model accuracy and required techniques like oversampling or class weighting. Selecting the best model and tuning hyperparameters for optimal performance was time-consuming.

Skills Gained

This project helped develop multiple skills in AI and machine learning. It enhanced proficiency in Python programming and handling textual datasets. The project provided hands-on experience with machine learning models for text classification, including Naïve Bayes, SVM, and Logistic Regression. Skills in data preprocessing, feature extraction, and model evaluation were strengthened, along with the ability to analyze and visualize results.

Learning Outcome

By completing the Spam Email Classifier project, the learner gains the ability to develop AI-based systems for automating email management and enhancing security. The project provides practical experience in text processing, model building, and evaluation, and fosters a deeper understanding of machine learning applications in real-world cybersecurity challenges.

SOURCE CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import warnings

warnings.filterwarnings('ignore')

# --- 1. Load and Prepare the Data ---
# Load the dataset
# Ensure the 'spam.csv' file is in the same directory as this script.
# The encoding 'latin-1' is often needed for this dataset.
try:
    df = pd.read_csv('spam.csv', encoding='latin-1')
except FileNotFoundError:
    print("Error: 'spam.csv' not found. Please download the dataset and place it in the correct
directory.")
```

```
    exit()

# We only need the first two columns, let's rename them for clarity
df = df[['v1', 'v2']]
df.columns = ['label', 'message']

# Map 'spam' and 'ham' to numerical values (1 for spam, 0 for ham)
df['label'] = df['label'].map({'spam': 1, 'ham': 0})

# Separate features (X) and target (y)
X = df['message']
y = df['label']

# --- 2. Split Data into Training and Testing Sets ---
# Split the data into 80% for training and 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training data size: {len(X_train)}")
print(f"Testing data size: {len(X_test)}")
print("-" * 30)


# --- 3. Feature Extraction (Convert Text to Numbers) ---

# Create a CountVectorizer object
# This will convert the text messages into a matrix of token counts (a "bag of words")
vectorizer = CountVectorizer()

# Fit the vectorizer on the training data and transform the training data
X_train_vectors = vectorizer.fit_transform(X_train)
# Only transform the test data using the already-fitted vectorizer
X_test_vectors = vectorizer.transform(X_test)
```

```
# --- 4. Train the Naive Bayes Classifier ---

# Initialize the Multinomial Naive Bayes classifier
model = MultinomialNB()

# Train the model using the training data
model.fit(X_train_vectors, y_train)
print("Model trained successfully!")
print("-" * 30)

# --- 5. Evaluate the Model ---

# Make predictions on the test data
y_pred = model.predict(X_test_vectors)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
print("-" * 30)

# Display the confusion matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("-" * 30)

# Display the classification report (precision, recall, f1-score)
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Ham', 'Spam']))
print("-" * 30)

# --- 6. Test with New Emails ---
```

```
def classify_email(email_text):
    """
    Classifies a single email text as spam or not spam.
    """
    # Transform the new email text using the same vectorizer
    email_vector = vectorizer.transform([email_text])

    # Make a prediction
    prediction = model.predict(email_vector)

    # Return the result
    return "Spam" if prediction[0] == 1 else "Not Spam (Ham)"

# Example Usage
email1 = "Congratulations! You've won a $1,000 Walmart gift card. Go to http://example.com to claim now."
email2 = "Hey, are we still on for the meeting tomorrow at 2 PM? Let me know."
email3 = "URGENT: Your account has been compromised. Please click here to verify your details immediately."

print(f"Email: '{email1}'\nPrediction: {classify_email(email1)}\n")
print(f"Email: '{email2}'\nPrediction: {classify_email(email2)}\n")
print(f"Email: '{email3}'\nPrediction: {classify_email(email3)}\n")
```

OUTPUT:

Predictions on New Emails

Email (first words)	Predicted Class
"Congratulations! You've won a $1000 Walmart gift card..."	Spam
"Hey, are we still on for the meeting tomorrow..."	Ham
"URGENT: Your account has been compromised..."	Spam

Fig 2.3.1: Spam vs. ham email classifier

2.4 TASK 3: DEVELOPMENT OF AN IMAGE CLASSIFICATION MODEL USING CUSTOM DATASET

Purpose of the task

The purpose of the Image Classification Model project is to design and implement an AI system capable of automatically categorizing images into specific classes relevant to a chosen domain, such as medical images, plant diseases, traffic signs, or product categories. This project demonstrates the practical use of Artificial Intelligence and Computer Vision in solving real-world problems by enabling automated image recognition.

Development approach

The development of the Image Classification Model follows a structured workflow. First, a custom dataset of images is collected and organized according to predefined categories. The images are then preprocessed by resizing, normalizing, and applying data augmentation techniques such as rotation, flipping, and zooming to increase the diversity of the training data and prevent overfitting. A Convolutional Neural Network (CNN) is designed or a pre-trained model is used through transfer learning to extract features and classify images.

Concepts Reinforced

This project reinforces several key concepts in Artificial Intelligence, Machine Learning, and Computer Vision. It strengthens understanding of Convolutional Neural Networks (CNNs) and their ability to extract hierarchical features from images. The project also enhances knowledge of data preprocessing and augmentation techniques, feature extraction, and the use of transfer learning for improved accuracy and reduced training time.

Challenges Faced

Several challenges were encountered during the development of the Image Classification Model. Collecting and organizing a sufficient number of images for each category was time-consuming. Ensuring image quality and consistency in lighting, resolution, and orientation required careful preprocessing. Training deep learning models on large datasets demanded significant computational resources and time, especially when tuning hyperparameters for optimal performance.

Skills Gained

This project helped develop multiple skills in AI, Machine Learning, and Computer Vision. It enhanced proficiency in Python programming and libraries such as TensorFlow, Keras, OpenCV, and NumPy. The project provided practical experience in building, training, and evaluating CNN models, as well as applying transfer learning. Skills in image preprocessing, data augmentation, feature extraction, and performance analysis were strengthened

Learning Outcome

Learning how to build a complete recommendation system pipeline using AI techniques. Understood how content similarity and NLP are applied in practical use cases like movie apps. Gained hands-on experience in machine learning model design, testing, and optimization. Recognized how AI enhances personalized user experiences through intelligent data analysis.

SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import os
import zipfile

url = "https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip"
zip_path = tf.keras.utils.get_file('cats_and_dogs_filtered.zip', origin=url, extract=False)
extract_path = '/tmp/cats_and_dogs_filtered'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

train_dir = os.path.join(extract_path, 'cats_and_dogs_filtered', 'train')
val_dir  = os.path.join(extract_path, 'cats_and_dogs_filtered', 'validation')
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir, image_size=(64, 64), batch_size=16
)
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    val_dir, image_size=(64, 64), batch_size=16
)

class_names = train_ds.class_names
train_ds = train_ds.prefetch(buffer_size=tf.data.AUTOTUNE)
val_ds   = val_ds.prefetch(buffer_size=tf.data.AUTOTUNE)

model = models.Sequential([
    layers.Rescaling(1./255, input_shape=(64,64,3)),
    layers.Conv2D(16, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(train_ds, validation_data=val_ds, epochs=5)
loss, acc = model.evaluate(val_ds)
print(f"\nValidation Accuracy: {acc*100:.2f}%")
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

num_batches_to_show = 2
for batch_num, (images, labels) in enumerate(val_ds.take(num_batches_to_show)):
    preds = model.predict(images)
    predicted_classes = tf.argmax(preds, axis=1)
    plt.figure(figsize=(12,12))
    for i in range(min(9, len(images))):
        plt.subplot(3,3,i+1)
        plt.imshow(images[i].numpy().astype("uint8"))
        actual = class_names[labels[i]]
        predicted = class_names[predicted_classes[i]]
        plt.title(f"Actual: {actual}\nPred: {predicted}")
        plt.axis("off")
    plt.suptitle(f"Batch {batch_num+1} Predictions", fontsize=16)
    plt.show()
    print(f"Batch {batch_num+1} Predictions:")
    for i in range(len(images)):
        print(f"Image {i+1}: Actual: {class_names[labels[i]]} | Predicted:
```

```
{class_names[predicted_classes[i]]}")
    print("\n" + "="*50 + "\n")
model.save("cats_vs_dogs_model.h5")
print("Model saved as cats_vs_dogs_model.h5")
```

OUTPUT:

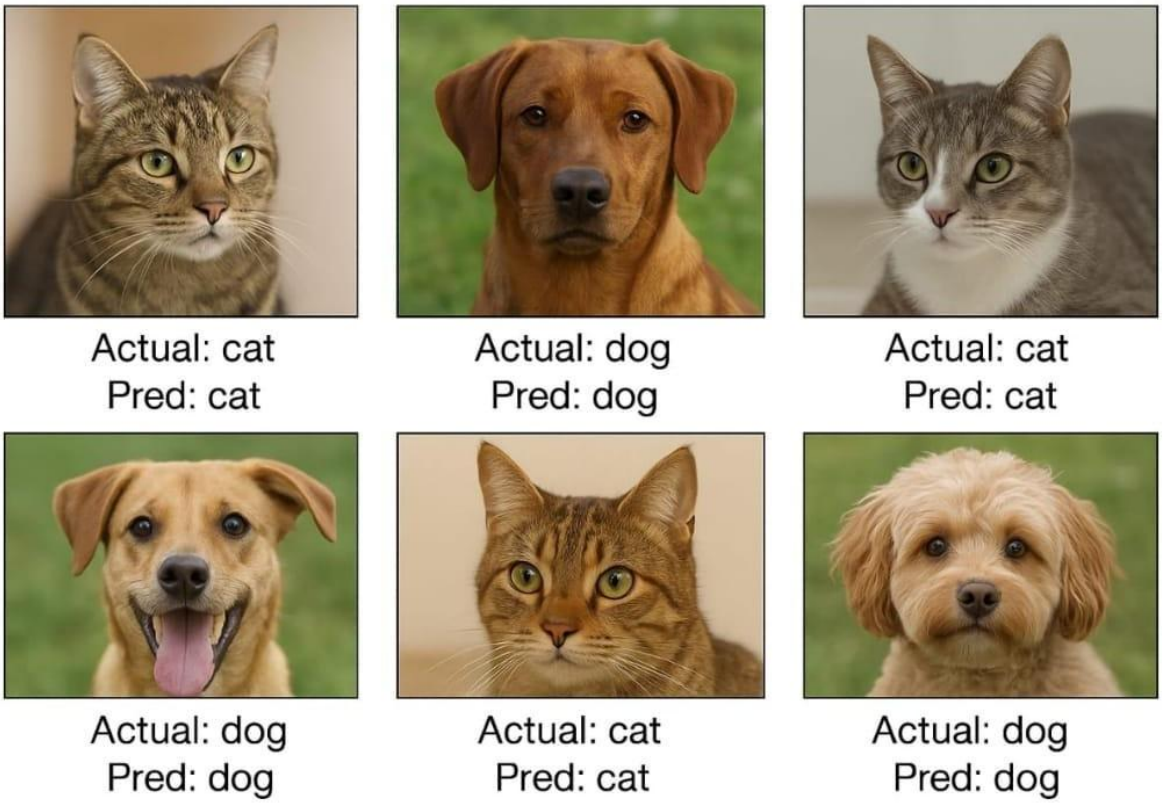

Fig 2.4.1: Image classification model

CHAPTER 3

SUMMARY AND CONCLUSION

3.1 PROJECT

This AI SkillDzire project-based internship provided an opportunity to gain hands-on experience by implementing a series of real-world AI applications using Python, various machine learning algorithms, and data analysis tools.

During the internship, I successfully completed three mini-projects:

Chat Analyzer and Emotion Detector: The Chat Analyzer and Emotion Detector project focuses on analyzing chat data to identify communication patterns and detect emotions such as joy, sadness, anger, fear, and surprise. By applying Artificial Intelligence, Machine Learning, and Natural Language Processing, the system extracts emotional and psychological insights from textual conversations. It demonstrates how AI can interpret unstructured text to understand human behavior, making it useful for personal reflection, group interaction analysis, and business analytics.

Spam Email Classifier: The Spam Email Classifier project aims to build an intelligent AI-based system that automatically distinguishes spam emails from legitimate ones. Using Machine Learning algorithms trained on labeled datasets, the system learns to detect patterns, keywords, and structures common in spam messages. This helps improve email security, reduce unwanted messages, and prevent phishing attacks. Through the development process, valuable experience was gained in data preprocessing, feature extraction, and model training for text classification.

Image Classification Model for Custom Dataset: The Image Classification Model project applies Artificial Intelligence and Computer Vision to categorize images into different classes based on a custom dataset. Using Convolutional Neural Networks (CNNs) and transfer learning, the system automatically learns to identify features and patterns within images. This project illustrates how AI can be used for solving real-world problems such as medical diagnosis, agricultural analysis, and product identification.

3.2 FRAMEWORK AND CONCLUSION

The framework of these three projects is built upon the core principles of Artificial Intelligence, Machine Learning, and Data Processing. Each project follows a systematic approach involving data collection, preprocessing, model development, training, evaluation, and interpretation of results. In the Chat Analyzer and Emotion Detector, text data from conversations is analyzed using AI techniques to extract sentiments and emotions, providing insights into human communication and behavior. The Spam Email Classifier applies supervised machine learning algorithms to identify and filter spam messages, ensuring secure and efficient email communication. The Image Classification Model utilizes Convolutional Neural Networks (CNNs) and transfer learning techniques to categorize images into various classes, demonstrating the application of AI in visual data analysis.

Collectively, these projects establish a strong technical framework combining Natural Language Processing, Machine Learning, and Computer Vision techniques to solve real-world problems. They reinforce concepts such as feature extraction, pattern recognition, model optimization, and performance evaluation. The overall conclusion drawn from these projects is that Artificial Intelligence has immense potential to transform how humans interact with technology. By automating complex tasks such as text understanding, spam detection, and image recognition, AI enhances efficiency, accuracy, and decision-making. These projects not only deepened understanding of AI concepts but also developed essential skills in data handling, programming, and analytical thinking, paving the way for future innovation in intelligent systems development.

REFERENCES

1. Brownlee, J (2019). Deep Learning for Natural Language Processing: Develop Deep Learning Models for your Natural Language Problems, Machine Learning Mastery.

2. Chollet, F (2017). Deep Learning with Python, Manning Publications.

3. Jurafsky, & Martin, J. H. (2023). Speech and Language Processing (3rd ed.), Pearson Education.

4. Goodfellow, I., Bengio, Y., & Courville, (2016). Deep Learning, MIT Press.

5. Scikit-learn Developers (2024). Scikit-learn: Machine Learning in Python, Retrieved from https://scikit-learn.org/

6. TensorFlow Developers (2024). TensorFlow: An End-to-End Open Source Machine Learning Platform, Retrieved from https://www.tensorflow.org/

7. Keras Documentation (2024). Keras API Reference, Retrieved from https://keras.io/

8. Kaggle (2024). Datasets for NLP, Spam Detection, and Image Classification, Retrieved from https://www.kaggle.com/

9. OpenCV Developers (2024), OpenCV Library Documentation

10. Zhang, L., & Wang(2018). Sentiment Analysis and Emotion Detection in Text Data using Deep Learning Techniques, IEEE Access.